



Alocação Dinâmica de Memória

Aula sobre Ponteiros com Structs em C/C++

Prof. Filipe Tôrres

30 de abril de 2026

Motivação

- Como representar entidades complexas na memória?
- Como acessar e modificar esses dados com eficiência?
- Ponteiros com structs são fundamentais para manipulação de dados reais.

Declaração de Struct com typedef

```
typedef struct {  
    int x;  
    int y;  
} Ponto;
```

```
Ponto p1;  
p1.x = 10;  
p1.y = 20;
```

Exercício 1 - Struct Básica

- Declare uma struct `Livro` com os campos:
 - `titulo` (string)
 - `autor` (string)
 - `ano` (inteiro)
- Preencha os campos de uma variável `livro1` e imprima os dados.

Exercício 1 - Struct Básica

```
#include <stdio.h>

typedef struct {
    char titulo[100];
    char autor[100];
    int ano;
} Livro;

int main() {
    Livro livro1;

    // Preenchendo os campos da variável livro1
    printf("Digite o titulo do livro: ");
    fgets(livro1.titulo, 100, stdin);

    printf("Digite o autor do livro: ");
    fgets(livro1.autor, 100, stdin);

    printf("Digite o ano do livro: ");
    scanf("%d", &livro1.ano);

    // Imprimindo os dados
    printf("\nDados do livro:\n");
    printf("Titulo: %s", livro1.titulo);
    printf("Autor: %s", livro1.autor);
    printf("Ano: %d\n", livro1.ano);

    return 0;
}
```

Ponteiros para Structs

```
Ponto *ptr; ptr = &p1;  
printf("%d", ptr->x); // ou (*ptr).x
```

- Use -> com ponteiros para structs: ptr->x = 10;
- Use . com variáveis comuns: p1.x = 10;

Exercício 2 - Acesso com Ponteiro

- Crie um ponteiro para a struct Livro
- Use -> para acessar e imprimir os campos

Exercício 2 - Acesso com Ponteiro

```
#include <stdio.h>

typedef struct {
    char titulo[100];
    char autor[100];
    int ano;
} Livro;

int main() {
    Livro livro1 = {"Dom Casmurro", "Machado de Assis", 1899};

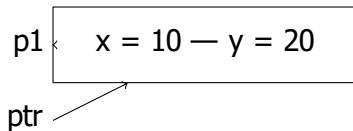
    Livro *ptr;

    ptr = &livro1;

    printf("Titulo: %s\n", ptr->titulo);
    printf("Autor: %s\n", ptr->autor);
    printf("Ano: %d\n", ptr->ano);

    return 0;
}
```

Visualização na Memória



Cadastro de Alunos

```
typedef struct {  
    char nome[50];  
    int matricula;  
    float nota;  
} Aluno;  
  
Aluno a1 = {"João", 123, 8.5};  
Aluno *ptr = &a1;  
printf("%s - %.2f", ptr->nome, ptr->nota);
```

Exercício 3 - Struct Aluno

- Crie a struct Aluno
- Atribua valores a uma variável
- Use ponteiros para acessar e imprimir os dados

Exercício 3 - Struct Aluno

```
#include <stdio.h>

typedef struct {
    char nome[50];
    int matricula;
    float nota;
} Aluno;

int main() {
    Aluno aluno1 = {"Joao", 12345, 8.5};

    Aluno *ptr;

    ptr = &aluno1;

    printf("Dados do aluno:\n");
    printf("Nome: %s\n", ptr->nome);
    printf("Matricula: %d\n", ptr->matricula);
    printf("Nota: %.2f\n", ptr->nota);

    return 0;
}
```

Structs em Arrays com Ponteiros

```
Aluno turma[3];
```

```
Aluno *p = turma; p->nota = 8.5;
```

- Ponteiro aponta para o primeiro elemento do array
- Use aritmética de ponteiros para navegar

Structs em Arrays com Ponteiros

```
#include <stdio.h>

typedef struct {
    char nome[50];
    int matricula;
    float nota;
} Aluno;

int main() {
    Aluno turma[3];

    Aluno *p = turma;

    p->matricula = 101;
    p->nota = 8.5;

    (p + 1)->matricula = 102;
    (p + 1)->nota = 7.0;

    (p + 2)->matricula = 103;
    (p + 2)->nota = 9.2;

    printf("Aluno 1 - Matricula: %d | Nota: %.2f\n", p->matricula, p->nota);
    printf("Aluno 2 - Matricula: %d | Nota: %.2f\n", (p + 1)->matricula, (p + 1)->nota);
    printf("Aluno 3 - Matricula: %d | Nota: %.2f\n", (p + 2)->matricula, (p + 2)->nota);

    return 0;
}
```

Exercício 4 - Vetor de Structs

- Crie um vetor de 3 structs Aluno
- Use ponteiros para preencher os dados

Exercício 4 - Vetor de Structs

```
#include <stdio.h>
#include <string.h>

typedef struct {
    char nome[50];
    int matricula;
    float nota;
} Aluno;

int main() {
    Aluno turma[3];

    Aluno *p = turma;

    strcpy(p->nome, "Joao");
    p->matricula = 101;
    p->nota = 8.5;

    strcpy((p + 1)->nome, "Maria");
    (p + 1)->matricula = 102;
    (p + 1)->nota = 9.0;

    strcpy((p + 2)->nome, "Carlos");
    (p + 2)->matricula = 103;
    (p + 2)->nota = 7.5;

    printf("Dados da turma:\n\n");

    for (int i = 0; i < 3; i++) {
        printf("Aluno %d\n", i + 1);
        printf("Nome: %s\n", (p + i)->nome);
        printf("Matricula: %d\n", (p + i)->matricula);
        printf("Nota: %.2f\n", (p + i)->nota);
    }

    return 0;
}
```

Resumo: . ou ->

- `p1.x` — variável do tipo struct
- `ptr->x` — ponteiro para struct
- `(*ptr).x` — equivalente a `ptr->x`

Desafio Final

```
typedef struct {  
    char nome[20];  
    float preco;  
} Produto;
```

// Complete o código:

```
Produto *p;
```

// Atribua valores e imprima usando ->

Conclusão

- Structs permitem organizar dados complexos
- Ponteiros evitam cópias e permitem manipulação eficiente
- Diferença clara entre . e ->